Package 'InferMultiModal'

November 7, 2026

Type Package

Title Structured Factor Regression with High-Dimensional Multimodal Data

Version 0.1.0

Author Xing Qin, Shuangge Ma, Mengyun Wu

Maintainer Xing Qin <qinxing@suibe.edu.cn>

Description This package is created for the practical application of our work 'Structured Factor Regression with High-Dimensional Multimodal Data'

License GPL-2

Encoding UTF-8

Imports GrFA,
ncvreg,
Matrix,
glmnet,
SIS,
MASS,
ACAT,
metap

Remotes yaowuliu/ACAT

RoxygenNote 7.3.3

Suggests knitr, rmarkdown

VignetteBuilder knitr

LazyData true

Contents

betatest 2
estimation 4
gammatest 6
predict.estimation 8

Index **10**

betatest *Hypothesis Test of the Effects of Idiosyncratic Residuals*

Description

Hypothesis Test of the Effects of Idiosyncratic Residuals

Usage

```
betatest(
  X,
  y,
  K,
  fit,
  rmax = 10,
  r0 = NULL,
  r = NULL,
  method = "gap",
  weight = TRUE,
  type = "BIC3",
  lambda.seq = NULL,
  alpha = 0.05,
  factor_res = NULL,
  boot_rep = 5,
  boot_each = 500,
  gamma_MCP = 3,
  seed_sis = 1,
  tune_sis = "ebic",
  tun_ncvreg = "CV"
)
```

Arguments

X	A list of the multi-modality data, each element is a data matrix of each group with dimension n*p_k.
y	A response vector.
K	The number of modalities.
fit	An object returned by [estimation()].
rmax	The maximum factor numbers of all modalities.
r0	The number of common factors. Default is NULL, the algorithm will automatically estimate the number of global factors. If you have prior information about the true number of global factors, you can set it manually.
r	The number of local factors in each group. Default is NULL, the algorithm will automatically estimate the number of local factors. If you have prior information, set it manually as an integer vector of length K (the number of groups).
method	The method used in the APM algorithm, default is gap, it can also be ic.
weight	The weight of each projection matrix in the APM algorithm, default is TRUE. if weight = FALSE, then simply calculate the mean of all projection matrices.

type	The method used in estimating the factor numbers in each modality initially, default is BIC3.
lambda.seq	A user supplied non-negative tuning parameters.
alpha	The significance level used in hypothesis testing. The default is 0.05.
factor_res	A list containing the following components: **Ghat** The supplied common factors. A numeric matrix or vector. **loading_G** A list consisting of the common factor loadings. Each element of the list corresponds to one common factor. **Fhat** The supplied specific factors. A numeric matrix or vector. **loading_F** A list consisting of the specific factor loadings. Each list element corresponds to one specific factor. **residual** A list consisting of the residuals.
boot_rep	Integer. The number of bootstrap repetitions (outer loop).
boot_each	Integer. The number of samples per repetition (inner loop).
gamma_MCP	The tuning parameter of the MCP penalty. Default is 3 for MCP.
seed_sis	An integer specifying the random seed used by the SIS method for noise variance estimation.
tune_sis	Method for tuning the regularization parameter of the penalized likelihood subproblems and the final model in the SIS noise variance estimation step. Options include "bic", "ebic", "aic", and "CV".
tun_ncvreg	Character string specifying the tuning method for the regression step. Options are "BIC" or "CV".

Value

A list containing the following components:

res A numeric indicator vector of length $1 + K + p$. The first element corresponds to the overall significance test, the next K elements correspond to the significance results for each modality, and the remaining p elements correspond to the significance results for each individual variable.

pvals A numeric vector of p-values with the same length ($1 + K + p$) as res. Each element corresponds to the respective test in res.

Examples

```
n=300
set.seed(2025)
K=4
p=1200
r_f=c(2,rep(2,K))
r_all=sum(r_f)
gamma1=0.5*rep(1,r_all)
pm=rep(p/K,K)
beta_0=c(rep(0,p))
# ------------------------#beta in the alternative with signal strength w rises from 0.05 to 0.2--------------
# beta_1=c(rep(0.05,2),rep(0,p/K-2),rep(0.05,2),rep(0,p/K-2),rep(0.05,2),rep(0,p/K-2),rep(0.05,2),rep(0,p/K
# beta_2=c(rep(0.1,2),rep(0,p/K-2),rep(0.1,2),rep(0,p/K-2),rep(0.1,2),rep(0,p/K-2),rep(0.1,2),rep(0,p/K-2))
# beta_3=c(rep(0.15,2),rep(0,p/K-2),rep(0.15,2),rep(0,p/K-2),rep(0.15,2),rep(0,p/K-2),rep(0.15,2),rep(0,p/K
# beta_4=c(rep(0.2,2),rep(0,p/K-2),rep(0.2,2),rep(0,p/K-2),rep(0.2,2),rep(0,p/K-2),rep(0.2,2),rep(0,p/K-2))
```

```
library(Matrix)
B_c<-matrix(rnorm(p*r_f[1]),nrow=p)
B=B_c
B_s<-list()
  for(k in 1:K){
    B_s[[k]]<-matrix(rnorm(pm[k]*r_f[k+1]),nrow=pm[k])
  }
block_diag <- as.matrix(bdiag(B_s[[1]], B_s[[2]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[3]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[4]]))
B=cbind(B_c,as.matrix(block_diag))
F<-matrix(rnorm(n*r_all),nrow=n)
V<-matrix(rnorm(n*p),nrow=n)
X=F%*%t(B)+V
ss=1
XX=list()
for(k in 1:K){
  XX[[k]]=X[,ss:(ss+pm[k]-1)]
  ss=ss+pm[k]
}
Y=F%*%gamma1+rnorm(n,0,1)
res_fac = estimation_fac(XX, K, nodewise_step=TRUE)
fit = estimation(XX,Y, K,res_fac =res_fac)
res <- betatest(XX,Y,K,fit=fit)
```

`estimation` *Estimation function for the proposed model.*

Description

Estimation function for the proposed model.

Usage

```
estimation(
  X,
  y,
  K,
  rmax = 10,
  r0 = NULL,
  r = NULL,
  method = "gap",
  weight = TRUE,
  type = "BIC3",
  lambda.seq = NULL,
  factor_res = NULL,
  res_fac,
  gamma_MCP = 3,
  tun_ncvreg = "CV"
)
```

Arguments

X	A list of the multi-modality data, each element is a data matrix of each group with dimension n*p_k.
y	A response vector.
K	The number of modalities.
rmax	The maximum factor numbers of all modalities.
r0	The number of common factors. Default is NULL, the algorithm will automatically estimate the number of global factors. If you have prior information about the true number of global factors, you can set it manually.
r	The number of local factors in each group. Default is NULL, the algorithm will automatically estimate the number of local factors. If you have prior information, set it manually as an integer vector of length K (the number of groups).
method	The method used in the APM algorithm, default is gap, it can also be ic.
weight	The weight of each projection matrix in the APM algorithm, default is TRUE. if weight = FALSE, then simply calculate the mean of all projection matrices.
type	The method used in estimating the factor numbers in each modality initially, default is BIC3.
lambda.seq	A user supplied non-negative tuning parameters.
factor_res	A list containing the following components: **Ghat** The supplied common factors. A numeric matrix or vector. **loading_G** A list consisting of the common factor loadings. Each element of the list corresponds to one common factor. **Fhat** The supplied specific factors. A numeric matrix or vector. **loading_F** A list consisting of the specific factor loadings. Each list element corresponds to one specific factor. **residual** A list consisting of the residuals.
res_fac	A list (or object) returned by estimation_fac, containing the estimated factor components.
gamma_MCP	The tuning parameter of the MCP penalty. Default is 3 for MCP.
tun_ncvreg	Character string specifying the tuning method for the regression step. Options are "BIC" or "CV".

Value

An object of class "estimation" containing the fitted model.

Examples

```
set.seed(1)
K=4
p=1200
library(Matrix)
r_f=c(2,rep(2,K))
r_all=sum(r_f)
pm=rep(p/K,K)
gamma1=rep(0.5,r_all)
beta=c(rep(0.5,2),rep(0,pm[1]-2),rep(0.5,2),rep(0,pm[2]-2),rep(0.5,2),rep(0,pm[3]-2),rep(0.5,2),rep(0,pm[4]
B_c<-matrix(rnorm(p*r_f[1]),nrow=p)
```

```
B=B_c
B_s<-list()
for(k in 1:K){
  B_s[[k]]<-matrix(rnorm(pm[k]*r_f[k+1]),nrow=pm[k])
}

block_diag <- as.matrix(bdiag(B_s[[1]], B_s[[2]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[3]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[4]]))
B=cbind(B_c,as.matrix(block_diag))
r<-0.3
n=ceiling(log(p)/(r^2/4^2))
dis=NULL
rr_hat=NULL
F<-matrix(rnorm(n*r_all),nrow=n)
V<-matrix(rnorm(n*p),nrow=n)
X=F%*%t(B)+V
Y=F%*%gamma1+V%*%beta+rnorm(n,mean=0,sd=1)
ss=1
XX=list()
for(k in 1:K){
  XX[[k]]=X[,ss:(ss+pm[k]-1)]
  ss=ss+pm[k]
}
res_fac<-estimation_fac(XX,K,method='gap')
fit<-estimation(XX,Y,K,res_fac=res_fac)
```

`gammatest` *Hypothesis Test of the Effects of Common and Modality-specific Factors.*

Description

Hypothesis Test of the Effects of Common and Modality-specific Factors.

Usage

```
gammatest(
  X,
  y,
  K,
  ratio1,
  ratio2,
  rmax = 10,
  r0 = NULL,
  r = NULL,
  method = "gap",
  weight = TRUE,
  type = "BIC3",
  lambda.seq = NULL,
  r_f = NULL,
  alpha = 0.05,
  factor_res = NULL,
```

```
  gamma_MCP = 3,
  seed_sis = 1,
  tune_sis = "ebic",
  tun_ncvreg = "CV"
)
```

Arguments

X	A list of the multi-modality data, each element is a data matrix of each group with dimension n*p_k.
y	A response vector.
K	The number of modalities.
ratio1	The exponent used in the sure screening step.
ratio2	The exponent used in the ANOVA test step.
rmax	The maximum number of factors across all modalities.
r0	The number of common factors. Default is NULL, the algorithm will automatically estimate the number of global factors. If you have prior information about the true number of global factors, you can set it manually.
r	The number of local factors in each group. Default is NULL, the algorithm will automatically estimate the number of local factors. If you have prior information, set it manually as an integer vector of length K (the number of groups).
method	The method used in the APM algorithm. The default is gap, it can also be ic.
weight	Logical. Whether to use weighted projection matrices in the APM algorithm. The default is TRUE. If FALSE, the algorithm simply calculates the mean of all projection matrices.
type	The method used to estimate the number of factors in each modality initially. The default is "BIC3".
lambda.seq	A user-supplied sequence of non-negative tuning parameters.
r_f	A vector specifying the number of factors in each modality. The default is NULL.
alpha	The significance level used in hypothesis testing. The default is 0.05.
factor_res	A list containing the following components: **Ghat** The supplied common factors. A numeric matrix or vector. **loading_G** A list consisting of the common factor loadings. Each element of the list corresponds to one common factor. **Fhat** The supplied specific factors. A numeric matrix or vector. **loading_F** A list consisting of the specific factor loadings. Each list element corresponds to one specific factor. **residual** A list consisting of the residuals.
gamma_MCP	The tuning parameter of the MCP penalty. Default is 3 for MCP.
seed_sis	An integer specifying the random seed used by the SIS method for noise variance estimation.
tune_sis	Method for tuning the regularization parameter of the penalized likelihood subproblems and the final model in the SIS noise variance estimation step. Options include "bic", "ebic", "aic", and "CV".
tun_ncvreg	Character string specifying the tuning method for the regression step. Options are "BIC" or "CV".

Value

A numeric vector containing the p-values testing the significance of the effects of common and modality-specific factors.

Examples

```
p <- 1200
n=300
set.seed(1)
K=4
r_f=c(2,rep(2,K))
r_all=sum(r_f)
pm=rep(p/K,K)
beta=c(rep(1,2),rep(0,pm[1]-2),rep(1,2),rep(0,pm[2]-2),rep(1,2),rep(0,pm[3]-2),rep(1,2),rep(0,pm[4]-2))
library(Matrix)
B_c<-matrix(rnorm(p*r_f[1],sd=1),nrow=p)
B=B_c
B_s<-list()
for(k in 1:K){
  B_s[[k]]<-matrix(rnorm(pm[k]*r_f[k+1],sd=1),nrow=pm[k])
}
block_diag <- as.matrix(bdiag(B_s[[1]], B_s[[2]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[3]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[4]]))
B=cbind(B_c,as.matrix(block_diag))
gamma1=c(rep(2,r_all))
set.seed(1)
F<-matrix(rnorm(n*r_all),nrow=n)
V<-matrix(rnorm(n*p),nrow=n)
X=F%*%t(B)+V #'design matrix
y=V%*%beta+rnorm(n,mean=0,sd=1)
y1=F%*%gamma1+y

ss=1
XX=list()
for(k in 1:K){
  XX[[k]]=X[,ss:(ss+pm[k]-1)]
  ss=ss+pm[k]
}
res=gammatest(XX,y,K,ratio1=0.9,ratio2=0.6,rmax=10,method='gap',weight=TRUE,lambda.seq=NULL,r_f=NULL,alpha=
```

`predict.estimation` *Prediction method for estimation objects*

Description

Defines the predict() S3 method for objects of class "estimation".

Usage

```
## S3 method for class 'estimation'
predict(object, newdata, ...)
```

Arguments

object	An object of class "estimation" (returned by estimation()).
newdata	A data frame containing the new predictor values.

Value

A numeric vector of predicted values.

Examples

```
set.seed(1)
K=4
p=1200
library(Matrix)
r_f=c(2,rep(2,K))
r_all=sum(r_f)
pm=rep(p/K,K)
gamma1=rep(0.5,r_all)#'c(0,0.3,0.5,0.5,0.5,0,0,0,0,0.5)
beta=c(rep(0.5,2),rep(0,pm[1]-2),rep(0.5,2),rep(0,pm[2]-2),rep(0.5,2),rep(0,pm[3]-2),rep(0.5,2),rep(0,pm[4]
B_c<-matrix(rnorm(p*r_f[1]),nrow=p)
B=B_c
B_s<-list()
for(k in 1:K){
  B_s[[k]]<-matrix(rnorm(pm[k]*r_f[k+1]),nrow=pm[k])
}

block_diag <- as.matrix(bdiag(B_s[[1]], B_s[[2]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[3]]))
block_diag <- as.matrix(bdiag(block_diag, B_s[[4]]))
B=cbind(B_c,as.matrix(block_diag))
n=ceiling(log(p)/(0.5^2/4^2))
F<-matrix(rnorm(n*r_all),nrow=n)
V<-matrix(rnorm(n*p),nrow=n)
X=F%*%t(B)+V
Y=F%*%gamma1+V%*%beta+rnorm(n,mean=0,sd=1)

ss=1
XX=list()
for(k in 1:K){
  XX[[k]]=X[,ss:(ss+pm[k]-1)]
  ss=ss+pm[k]
}
res_fac<-estimation_fac(XX,K,method='gap')
fit<-estimation(XX,Y,K,res_fac=res_fac)
n_test <- 100
F_test <- matrix(rnorm(n_test * r_all), nrow = n_test)
V_test <- matrix(rnorm(n_test * p), nrow = n_test)
X_test <- F_test %*% t(B) + V_test
Y_test <- F_test%*%gamma1+V_test%*%beta+rnorm(n_test,mean=0,sd=1)
y_pred <- predict(fit,X_test)
```

Index

betatest, 2

estimation, 4

gammatest, 6

predict.estimation, 8